# Wrap or Rewrite? Reviving the 1989 *GlobalMinimum* Bayesian Global-Optimization Library for R and Python

AUDRIS MOCKUS, University of Tennessee, USA

The GlobalMinimum library of Jonas Mockus (1989) implements some of the earliest practical Bayesian global-optimization methods. We modernize this Fortran 66/77 codebase along both standard routes: wrapping the original compiled routines behind R and Python packages, and translating the core methods to Rust. First, we release packages in which every optimizer runs the genuine 1989 algorithms, with callback objectives, deterministic seeding, evaluation traces, and validation that retrofits memory- and I/O-safety onto COMMON-block Fortran. Second, we dissect a translation-fidelity incident: the Rust port ran up to 12x slower than the wrapped Fortran—not from foreign-function overhead, but from two dropped pruning rules whose restoration recovers bitwise trajectory agreement and speed parity. The same audit surfaced transposed test-function matrices and an uninitialized read in the upstream code itself. Third, we benchmark the 1989 methods against modern optimizers in both ecosystems (78,000 runs, externally validated on BBOB/COCO, bitwise-replicated across languages): the coordinate Wiener-model search EXKOR statistically ties SHGO, dual annealing, and GenSA on classical test functions but drops to mid-field on BBOB's non-separable landscapes, and a compound method built from the library's own components trades aggregate solve rate for markedly fewer total failures. All code and data are available for replication.

CCS Concepts: • **Mathematics of computing** → **Nonconvex optimization**; **Mathematical software performance**; • **Software and its engineering** → *Software verification and validation.*

Additional Key Words and Phrases: global optimization, Bayesian optimization, legacy software, Fortran, Rust, translation fidelity, benchmarking, performance profiles

**ACM Reference Format:**
Audris Mockus. 2026. Wrap or Rewrite? Reviving the 1989 *GlobalMinimum* Bayesian Global-Optimization Library for R and Python. *ACM Trans. Math. Softw.* 0, 0, Article 0 ( 2026), 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 INTRODUCTION

Between the first expected-improvement formulations of Kushner [15] and the modern Gaussian-process machinery popularized by EGO [13] and surveyed in [5, 24] lies a substantial body of practical Bayesian global-optimization software developed by Jonas Mockus and collaborators in Vilnius from the 1960s through the 1980s [16, 18]. Its concrete artifact, the *MINIMUM*/*GlobalMinimum* Fortran library [17] distributed with the 1989 monograph [16], packages fourteen optimization and analysis routines: the one-step Bayesian method `BAYES1`, a Wiener-model one-dimensional global search `EXTR` and its coordinate-descent extension `EXKOR`, uniform-search and clustering multistart methods (`UNT`, `GLOPT`), LP-$\tau$ low-discrepancy search `LPMIN` [25], local methods (`MIVAR4`, `FLEXI`, `REQP`), Monte-Carlo baselines (`MIG1`, `MIG2`), and variable-screening procedures (`ANAL1`, `ANAL2`).

This library is scientifically interesting today for two reasons. As *methods*, its algorithms are extremely inexpensive surrogate-based optimizers: the `BAYES1` planner selects each new point by

Author's address: Audris Mockus, University of Tennessee, Knoxville, TN, USA, audris@utk.edu.


0098-3500/2026/0-ART0 $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

maximizing a closed-form acquisition over the full evaluation history at $O(\ell n)$ cost per candidate, in contrast to the $O(\ell^3)$ Gaussian-process algebra of modern Bayesian optimization. How such methods fare against contemporary default choices (differential evolution, generalized simulated annealing, DIRECT, CMA-ES) under matched budgets is an empirical question that, to our knowledge, has not been answered with modern benchmarking methodology. As *software*, the library is a representative specimen of scientific legacy code—fixed-form Fortran with COMMON-block state, link-time objective binding, single-precision utilities promoted by compiler flag, and console I/O woven into numerical routines—and thus a concrete case study in the perennial engineering question: **wrap the original, or rewrite it in a modern language?**

We did both, and instrumented the difference. This paper reports:

- **Packages** (Section 3). A CRAN-style R package in which all fourteen routines run the *original* compiled Fortran, and a PyPI-style Python package wrapping the twelve optimization routines (the two screening procedures remain translation-backed in Python). A small trampoline (the upstream objective `FI(X,N)` is a link-time symbol, not an argument) and a language-agnostic C shim provide interpreter callbacks, compiled built-in objectives, evaluation tracing, deterministic seeding of the library's lagged-Fibonacci generator, and pre-call validation that makes the packages memory-safe against every documented limit of the 1989 working arrays. A Rust crate hosts both the translation and, via the same shim, the wrapped original; the Python extension exposes the two side by side as `backend="fortran"` and `backend="rust"`.
- **A translation-fidelity study** (Section 4). An early observation that "wrapping Fortran is about 4× faster than the Rust translation" resisted the obvious explanation. By separating foreign-function call overhead, interpreter-callback cost, and algorithm-internal cost, we show the overheads are microseconds and nearly backend-independent, while the translated `BAYES1` planner was 1.7× ($n$=2) to 12× ($n$=20) slower because the port dropped two pruning rules of the original acquisition scan. Restoring them yields bitwise trajectory agreement with the 1989 binary and runtime parity. We also document independent correctness bugs found only because the original was executable: re-implemented Shekel and Hartmann test matrices were transposed relative to the Fortran `DATA` statements in both prior re-implementations (all five coefficient families in the pure-R port, two of five in the Rust port).
- **A cross-ecosystem benchmark** (Sections 5 and 6). A COCO-style [7] suite (16 test families, dimensions 2–10, 15 shifted instances each, budgets of $25n$ and $100n$ evaluations, identical bit-level instances in R and Python) comparing seven wrapped 1989 methods against SciPy (differential evolution [26], dual annealing [29, 32], DIRECT [6, 12], SHGO [4]), NLopt [11] (DIRECT-L, CRS2 [14], MLSL [21], ISRES [22]), CMA-ES [8], Gaussian-process expected improvement (scikit-optimize [9]), and the R packages GenSA [31], DEoptim [20], and GA [23], evaluated with performance profiles [3] and data profiles [19], and externally validated on the official BBOB suite of the COCO platform [7] (24 functions, 43,920 additional runs).
- **A compound 1989 method** (`EXKOR/A2`, Sections 5 and 6), answering the question whether the historic components compose into an improved method: LP-$\tau$ design, `ANAL2` eigenframe estimation, and `EXKOR` search, combined unchanged under a simple influence-ratio guard. The compound repairs four of five of `EXKOR`'s total-failure families and improves its mean rank in both ecosystems while conceding a statistical tie in aggregate solve rate—evidence that composition yields robustness rather than throughput, and that the 1989 interfaces' inability to warm-start burdens any composition.

Beyond the specific library, the paper argues a methodological point for research software engineering: *when modernizing numerical legacy code, keep the original runnable and compare against it continuously.* Every consequential defect we found—performance-relevant pruning rules silently dropped, test-function coefficient matrices transposed, a latent uninitialized-variable bug in the upstream double-precision tree itself—was detected only through executable-to-executable comparison, not code review. The lesson is timely rather than antiquarian: large-language-model–assisted translation is currently industrializing exactly the rewrite route studied here, at a scale where nobody will hand-audit the output. Our failure taxonomy (complexity-relevant control flow discarded as incidental; array layouts transposed across language conventions; both invisible to review and to unit tests of summary values) and the discipline that catches it (full-trajectory parity against the executable original) transfer directly to that setting.

## 2 THE GLOBALMINIMUM LIBRARY

### 2.1 Methods

We briefly describe the routines wrapped by our packages; the monograph [16] gives derivations, and [28, 33] situate them historically.

*One-step Bayesian search (*`BAYES1`*).* After $L_0$ initial evaluations at LP-$\tau$ (Sobol-type) points, each subsequent point maximizes a closed-form acquisition: for candidate $x$ with history $(x_i, y_i)_{i\le\ell}$ and incumbent $y^*$,

$$\alpha(x) \;=\; \min_{i\le\ell} \; \frac{\|x - x_i\|^2}{y_i - y^* + \varepsilon}, \tag{1}$$

maximized over a Monte-Carlo candidate set of adaptive size $m = \min(50n, \max(10n, \ell n))$ drawn from the library's additive lagged-Fibonacci generator (`ATS`). Criterion (1) is a distance-to-merit ratio: a point is attractive when it is far from previously evaluated points whose values are close to the incumbent. It is the asymptotic simplification of the one-step Bayesian decision rule derived in [18], evaluable in $O(\ell n)$ without any matrix algebra. Two pruning rules make the scan cheap in practice, and both matter for Section 4: the inner distance accumulation aborts once the partial sum exceeds a threshold, and the scan aborts entirely once $\alpha$ falls below the running second-best candidate score (maintained across candidates in COMMON storage).

*Wiener-model one-dimensional search (*`EXTR`, `EXKOR`*).*. `EXTR` models a one-dimensional objective as a Wiener process, evaluating where the conditional probability of improving on the incumbent is largest, with a stopping rule at estimated posterior confidence 0.95; `EXKOR` applies it coordinate-wise. These predate, and structurally resemble, modern one-dimensional Bayesian-optimization schemes.

*Multistart and deterministic methods.* `UNT` couples uniform random starts with a Wiener-model uncertainty estimate to decide which local minima to pursue; `GLOPT` clusters promising samples and searches within clusters, in the spirit of [21]; `LPMIN` searches LP-$\tau$ low-discrepancy designs, optionally after a factor-analysis phase that orders variables by influence; `MIG1`/`MIG2` are Monte-Carlo baselines. Local methods comprise a variable-metric quasi-Newton code with finite-difference gradients (`MIVAR4`), a flexible-tolerance constrained simplex (`FLEXI`), and recursive quadratic programming (`REQP`). `ANAL1`/`ANAL2` screen variables by harmonic and covariance-eigenstructure analysis of an evaluated design.

### 2.2 Software archaeology

The distribution contains a single-precision tree and a hand-converted `REAL*8` tree. The conversion is instructive about pre-IEEE portability practice: algorithm files received explicit `IMPLICIT`

REAL*8 declarations, while the shared utilities (ATS, LPTAU, machine constants, test objectives) remained default-REAL, relying on an f77 -r8-style promotion flag. On x87 hardware a REAL function result read as REAL*8 happened to work (results returned at extended precision on the floating-point stack); on the x86-64 SSE ABI it silently returns garbage, so a faithful modern build *must* reproduce the promotion (-fdefault-real-8 -fdefault-double-8) or convert the utilities explicitly (we do both, converting the utility file and flagging the rest). The audit also exposed genuine bugs in the upstream code itself. MIVAR4's double-precision line search renamed its machine-constant bounds at the two sites where they are computed (SRMIN → SDMIN) but not at the nine sites where they are used, leaving the used variables implicitly zero; our vendored copy restores the intended values. In EXKOR's private copy of the one-dimensional engine, a degenerate parabola fit on the first pass through the local-optimization block stores the interpolation point before it has ever been assigned—an uninitialized read flagged by -Wmaybe-uninitialized during our determinism audit; tellingly, the standalone EXTR contains the repaired form of the same statement, so the defect was found and fixed upstream in one copy of the duplicated code but not the other. Our vendored copies initialize the point to the current simplex middle, which the degenerate branch otherwise re-stores idempotently. Two pairs of files define identical private helpers (COR/ALNORM in both extr.f and exkor.f; FAKTKK in both lpmin.f and anal2.f), which forbids linking the library into one shared object without renaming—acceptable in 1989 where each program linked one optimizer, fatal for a package that must load all of them.

Three interface properties shape any wrapper design: the objective FI(X,N) and constraint routine CONSTR(X,NX,R,K8) are *link-time symbols*, not arguments; control parameters arrive through offset-indexed IPAR(30)/PAR(30) arrays whose validation failures print to a Fortran output unit and set a COMMON flag; and all state, including the random-generator state and evaluation histories capped at fixed sizes (dimension $\leq 20$, $\leq 1000$ evaluations for the Bayesian routines), lives in COMMON blocks.

## 3 PACKAGE DESIGN

### 3.1 Architecture

Both packages share a three-layer design (Figure 1 schematically): the pristine vendored Fortran (four documented, mechanically applied source patches: the MIVAR4 repair, two symbol renames, and one DIMENSION statement that listed scalars, rejected by modern compilers); a Fortran trampoline defining FI and CONSTR to forward to C entry points; and a language-agnostic C shim that dispatches those calls to either a registered host-language callback or one of the eight compiled built-in test objectives, while counting evaluations, optionally recording the full evaluation trace, and exposing get/set access to the 15-word ATS generator state. Wrappers validate every documented limit *before* entering Fortran, so the upstream error branches—which would write to standard output and, in 1989 style, poke a COMMON flag—are unreachable; the printing parameter is pinned to silent.

Seeding deserves a note. The library initializes ATS from a BLOCK DATA constant, so a fresh 1989 process was deterministic, but state persisted *across* calls within a process, making repeated runs irreproducible. Our wrappers reset the state to the canonical constants before each run by default—every call reproduces a fresh-process run of the original code—and accept an integer seed that derives a perturbed state through a Park–Miller generator, giving properly replicated stochastic runs (the benchmark of Section 5 uses this). We found and fixed the opposite defect in the repository's earlier ad-hoc harness: optimizers silently ignored the seed, so "30-seed" summaries of the deterministic methods aggregated thirty identical runs against genuinely varying competitors.

```
R / Python / Rust API      (validation, seeding, results, docs, tests)
        |  .Call / pyo3 / extern "C"
C shim  gm_shim.c          (callback registry, builtins, trace, evals, ATS)
        |  gmcall_ / gmcon_
Fortran trampoline gm_fi.f (FI, CONSTR -> C; GMATSE/GMATGE state access)
        |  link-time
GlobalMinimum real.8       (14 routines, vendored + 4 documented patches)
```

Fig. 1. Layered wrapper architecture shared by the R package, the Python extension, and the Rust crate.

### 3.2 The R package

globalopt for R compiles the Fortran into the package shared object (standard `R CMD` toolchain; no external build steps). All fourteen routines are exposed with roxygen documentation, a vignette, and a testthat suite that pins LP-$\tau$ points, built-in objective values, and full `BAYES1`/`MIG2` runs to reference values obtained from a standalone C driver linked directly against the library. Objectives are R closures (called through the shim with error trapping; an R-level error or non-finite value aborts the search safely) or built-in names (`"furasn"`, `"fush5"`, ...) that keep the entire run in compiled code. Results carry the trace (points matrix, values), the exact evaluation count from the shim, and the Fortran termination code. A pure-R reference implementation of the core methods is retained as `backend="reference"`—pedagogically useful, and the measured base case for Section 4's interpreter comparison. `R CMD check` passes with one expected note (the compiled code retains Fortran `WRITE` statements on unreachable error paths) and one portability warning for the promotion flags, both documented.

### 3.3 The Python package and Rust crate

The Rust crate hosts the translation (`mig1`, `mig2`, `bayes1`, and modernized variants of the remaining methods) and, behind a `fortran` feature, the wrapped original: `build.rs` compiles the vendored sources with gfortran and links them with the same C shim, exposing safe wrappers that validate limits, serialize calls through a mutex (COMMON state is global), and return the same result struct as the translation. The Python extension (pyo3/maturin) publishes both sides as `backend="rust"` and `backend="fortran"`; Python callables are invoked through identical conversion machinery in either case, which is what makes the overhead comparison of Section 4 clean. The wheel bundles `libgfortran`, so end users need no Fortran toolchain.

## 4 WRAP VERSUS REWRITE: A FIDELITY STUDY

### 4.1 The "4×" anomaly

Ad-hoc timings in the project's history suggested the wrapped Fortran `BAYES1` ran roughly four times faster than the Rust translation, inviting explanations about foreign-function or interpreter overhead. We decomposed the difference with three controlled experiments on the same host (all scripts and raw data ship with the repository):

(1) **Call and callback overhead** (algorithm cost $\approx$ 0): `MIG2`, 1000 evaluations of the built-in `furasn` objective. Per evaluation, the wrapped Fortran costs 0.5–1.7 $\mu$s and the Rust translation 0.8–2.3 $\mu$s across dimensions 2–20. Supplying the objective as a Python callable adds 1–3 $\mu$s per evaluation *to either backend equally*; an R closure adds 2–4 $\mu$s. No 4× lives here.

(2) **Algorithm-dominated runs**: BAYES1, 1000 evaluations. Before our fix the Rust translation took 0.32 s, 3.4 s, and 13.9 s at $n = 2, 10, 20$ against 0.18 s, 1.02 s, and 1.16 s for the wrapped Fortran measured in the same session—a *dimension-dependent* 1.7×, 3.4×, 12× gap, incompatible with constant-per-call overhead.
(3) **Interpreter base case**: the pure-R reference implementation of the same algorithm takes 55–200 s, i.e. 84× ($n$=10, linearly extrapolated and therefore understated) to 465× ($n$=2, measured directly) the wrapped Fortran, dwarfing every wrapping-related effect.

### 4.2 Diagnosis and repair

The dimension dependence pointed into the acquisition scan (1). The upstream FIAP1 contains two pruning rules that a line-by-line translation had discarded as incidental control flow (GOTO-dense, COMMON-coupled): (i) the inner squared-distance accumulation aborts as soon as the partial sum reaches a threshold that certifies the observation cannot improve the score — the work saved grows linearly with dimension, matching the observed scaling; and (ii) the whole history scan aborts when the score drops below the running second-best across candidates, a bound communicated via COMMON from the candidate loop (MIG2F2), which the translation had replaced with a materialize-all-then-sort structure that also allocated per candidate. Restoring both rules—by rewriting the planner in the original's streaming two-best form—made the Rust translation match the wrapped original *bitwise* on 168 of 200 trace values on the reference problem (the remainder differ by one unit in the last place, gfortran versus Rust standard-library cosine) with runtime ratios of 1.31, 1.12, and 1.02 at $n = 2, 10, 20$; the residual low-dimension difference is per-candidate allocation, not algorithm.

### 4.3 Correctness hazards of rewriting

The same executable-to-executable audit caught quiet correctness bugs. The Shekel and Hartmann coefficient matrices appear in the Fortran as column-major DATA streams; both the pure-R and (for two of five matrices) the Rust re-implementations had refilled them row-wise, producing plausible-looking but wrong test functions—the transposed Shekel-7 still has minima around −10 in the right region. Because the compiled originals were callable next to the rewrites, a one-line test (same trajectory under built-in versus re-implemented objective) exposed the transpositions immediately. We take from this a simple discipline for numerical modernization: *keep the legacy binary runnable inside the test suite; parity of full trajectories, not of summary statistics, is the assertion of record.*

## 5 BENCHMARK DESIGN

*Problems and instances.* Eight scalable families (sphere, Rosenbrock, Rastrigin, Ackley, Griewank, Lévy, Schwefel, Zakharov) at $n \in \{2, 5, 10\}$ and eight fixed-dimension classics (Branin, Goldstein–Price, six-hump camel, Shekel-5/7/10, Hartmann-3/6) [27], each in 15 instances formed by seeded coordinate shifts of up to ±5% of the box (COCO style [7]), normalized so every instance has optimal value exactly 0. Instance generation uses a Park–Miller stream shared bit-for-bit by the R and Python harnesses, verified identical across languages.

*Protocol.* Budgets of $25n$ and $100n$ evaluations. Every solver sees the objective only through a recording wrapper that counts evaluations, tracks the best value within budget, and records the first evaluation index reaching gap tolerances $10^{-1}, \ldots, 10^{-8}$; solvers that overrun their nominal budget are truncated at analysis time (SciPy's SHGO is the only material offender). One run per (method, instance, budget); variation across the 15 instances provides replication, with stochastic solvers additionally seeded per instance (including the wrapped 1989 methods, through the ATS state described in Section 3).

*Methods.* Python: seven wrapped 1989 methods (BAYES1, MIG2, GLOPT, UNT, LPMIN, EXKOR, LBAYES), the repaired Rust translations of BAYES1/MIG2, SciPy differential evolution, dual annealing, DIRECT, SHGO, NLopt DIRECT-L, CRS2, MLSL, ISRES, CMA-ES (pycma), Gaussian-process Bayesian optimization with expected improvement (scikit-optimize; the modern realization of the program BAYES1 approximates), and uniform random search as control. The GP baseline runs only on the $25n$ tier at $n \leq 6$: its cubic-cost model refits already average tens of seconds per run there, which is also its natural operating regime. R: the same seven wrapped methods, GenSA, DEoptim, GA, NLopt via nloptr (DIRECT-L, CRS2, ISRES), and random search. The wrapped methods inherit the 1989 array caps (e.g. 1000 evaluations for BAYES1, 500 for UNT/EXKOR); where the budget exceeds a cap the method simply stops early and its profile flattens, which we report as-is: the caps are part of the artifact under study.

*Metrics.* Dolan–Moré performance profiles [3] on evaluations-to-target and Moré–Wild data profiles [19] at gap tolerances $10^{-2}$ and $10^{-4}$; median terminal gap tables; wall-clock time per run (cheap objectives, so time exposes solver-internal and FFI cost); and the FFI microbenchmarks of Section 4. Profiles and rank summaries are descriptive; for the six headline pairwise comparisons quoted in Section 6 we additionally run exact McNemar tests on the paired solved/unsolved outcomes over the 480 instances (all quoted $p$-values remain significant, or not, under Holm–Bonferroni correction across the six planned tests).

*A compound 1989 method (EXKOR/A2).* The component measurements pose a research question: *can the historic components be composed, unchanged, into an improved method?* The library supplies the parts: LP-$\tau$ designs, ANAL2's eigenframe estimate, and EXKOR's coordinate search. EXKOR/A2 spends 10% of the budget on an LP-$\tau$ design, estimates the eigenframe with ANAL2, runs plain EXKOR on the full box for 45%, and spends the final 45% restarting EXKOR at the incumbent—in the rotated frame when the dominant eigen-direction explains at least 1.3× the influence of the best original axis, in the original axes otherwise. The guard matters: an initial always-rotate variant repaired EXKOR's failures on linearly coupled problems but destroyed separable ones (rotating a separable function makes it non-separable), and a fixed 25% design fraction consumed budget on every problem; we report the portfolio form above and disclose that it is the second of two design iterations, both scripted in the repository.

*BBOB validation.* Because a fixed collection of classical test functions can encode unintended structure (four of our families even ship with the library), we replicate the Python-side experiment on the official BBOB suite via cocoex [7]: all 24 functions, dimensions $\{2, 5, 10\}$, instances 1–15 (1,080 problem instances), the same recorder protocol, budget tiers, seeding, and method set. Instance optima are recovered from the suite's optimizer locations so the same gap-based analysis applies unchanged; runs were sharded across three hosts.

## 6 RESULTS

The sweep comprises 19,560 Python runs (including the 360-run GP-Bayesian-optimization tier) and 14,400 R runs on the classical suite, plus 43,920 BBOB runs (Section 6.2); no run failed except the intentional SHGO exclusions above $n = 5$ — where its simplicial machinery takes minutes and gigabytes per run — which are recorded as failures and count against it. Raw CSVs, the analysis script, and the exact numbers below ship with the repository.

### 6.1 Where the 1989 methods stand

Figure 2 shows performance profiles at gap $10^{-2}$ under the $100n$ budget; Figure 3 (left) the corresponding data profile. Three regimes are visible.

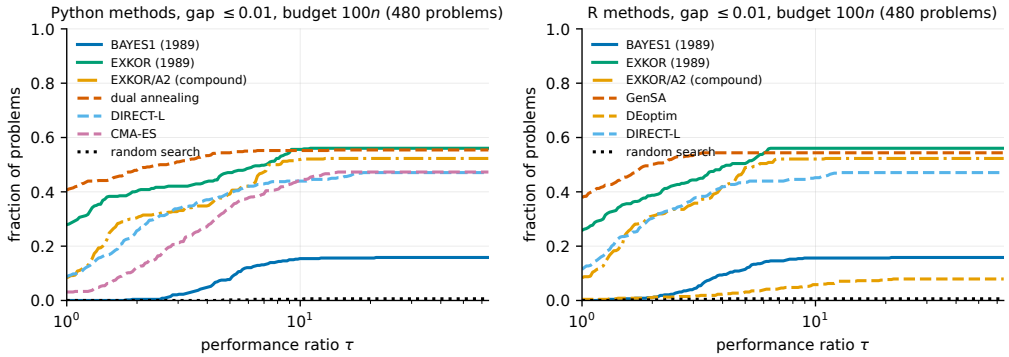


Fig. 2. Dolan–Moré performance profiles on evaluations to reach gap $10^{-2}$ within the $100n$ budget (480 problem instances; unsolved instances count in the denominator). Solid lines: wrapped 1989 methods; dashed: modern competitors; dotted: random search.

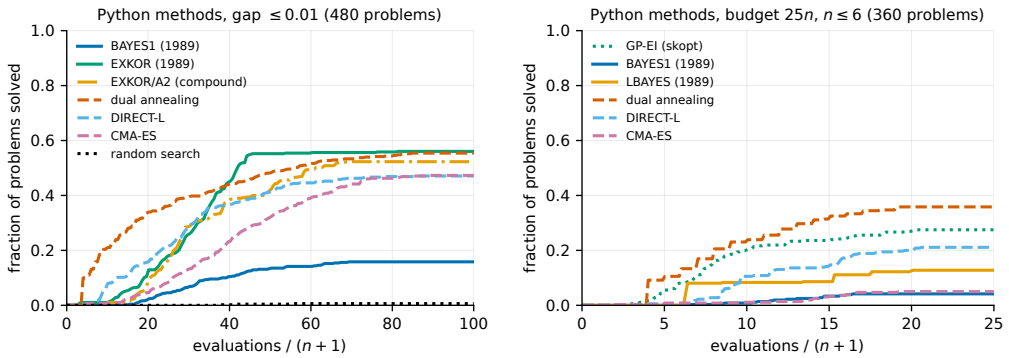


Fig. 3. Moré–Wild data profiles (fraction of instances solved to gap $10^{-2}$ versus evaluations/$(n+1)$), Python methods. Left: the $100n$ budget. Right: the $25n$ budget restricted to $n \leq 6$, the tier on which the GP-EI baseline runs.

*Generous budgets (100n).* The wrapped methods produce bitwise-identical sweeps in R and Python (same solve sets, discordant counts, and $p$-values — itself a strong wrapper-fidelity check), so we quote their numbers once. The coordinate Wiener-model search `EXKOR` is the strongest pure 1989 method and competitive with everything we ran: it solves 56.0% of instances within budget — statistically tied with SHGO (56.2%) and dual annealing (55.4%; 140 versus 137 discordant instances, McNemar $p = 0.90$), tied with GenSA (54.4%, 133 versus 125, $p = 0.66$), and ahead of CMA-ES (47.3%, $p = 0.006$) and DIRECT-L (47.1%, $p = 0.003$, identically in both ecosystems). Tightening the tolerance to $10^{-4}$ preserves the picture (SHGO 54.2%, dual annealing 52.9%, `EXKOR` 50.2%). `EXKOR`'s profile is sharply bimodal across problem families, exactly as its structure predicts: on separable or coordinate-aligned landscapes it is essentially perfect (100% of Hartmann-6, Lévy, Schwefel, six-hump camel, and sphere instances; 91% Rastrigin; 82% Ackley), while on coupled valleys it solves nothing (0% on Rosenbrock, Branin, Goldstein–Price, and Zakharov), and on Hartmann-3 its coordinate-wise moves descend into the secondary basin on every instance (a constant terminal gap of 0.773). Averaged ranks over all problem groups therefore place it eighth of twenty in

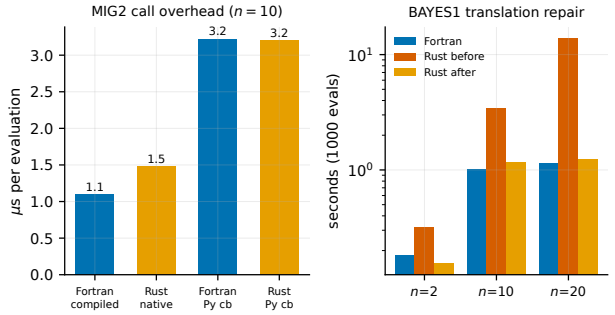


Fig. 4. The overhead/fidelity decomposition of Section 4: per-evaluation cost of the overhead-dominated MIG2 ($n$=10), and BAYES1 runtime before/after the pruning repair.

Python (mean rank 6.7 versus 4.8 for DIRECT-L) and fourth of fifteen in R (4.8, behind GenSA at 2.7, DIRECT-L at 3.4, and its own compound variant), ahead of CRS2, ISRES, DEoptim, and GA in both ecosystems.

*Tight budgets (25$n$).* Small-budget behavior reverses the order: dual annealing (31.0%) and GenSA (28.5%) dominate, the multistart LBAYES is the best 1989 method (12.7%, third overall in R at this budget), and EXKOR (4.4%) has not yet completed its first coordinate cycles. The one-step Bayesian BAYES1 sits mid-field at both budgets (15.8% at 100$n$); its fixed candidate-sampling acquisition is dominated by modern annealing schedules on these landscapes, though it beats CRS2, ISRES, and the differential-evolution and genetic-algorithm packages — implementations published one to two decades later — under the same protocol.

*Against modern Bayesian optimization.* On the tier where GP-based expected improvement is computationally feasible (25$n$ budget, $n \leq 6$; 360 instances), it solves 27.5% of instances — far ahead of its 1989 ancestors BAYES1 (4.2%, McNemar $p < 10^{-21}$) and LBAYES (12.8%, $p < 10^{-10}$), and behind only dual annealing (35.8%, $p = 8 \times 10^{-4}$). Figure 3 (right) shows the full data profile. The quality gap is what thirty-five years of surrogate and acquisition development bought; the counterweight is solver-internal time: median 77 s per 25$n$-budget run for GP-EI versus 9 ms for BAYES1 on the same tier — nearly four orders of magnitude — which is exactly the inexpensive-acquisition, many-evaluations regime the 1989 design targeted.

*The compound method.* EXKOR/A2 answers its research question with a precise "yes, for robustness — no, for aggregate throughput." At 100$n$ it repairs four of plain EXKOR's five zero-solve families (Hartmann-3 0 → 100%, Zakharov 0 → 33%, Goldstein–Price 0 → 27%, Rosenbrock 0 → 13%; only Branin resists), cutting zero-solve families from five to two, and improves the all-problems mean rank in both ecosystems (6.3 versus 6.7 in Python; 4.2 versus 4.8 in R, where it overtakes plain EXKOR for third place). The concession appears exactly where the budget split predicts: on multimodal separable problems that plain EXKOR solved with its full budget (Rastrigin −78, six-hump camel −60, Schwefel −24 percentage points), for an aggregate solve rate of 52.3% versus 56.0% (41 versus 59 discordant, $p = 0.089$ — a statistical tie, identical in both languages). On BBOB it is nominally ahead (15.3% versus 14.6%, $p = 0.32$). At the 25$n$ tier its three-phase structure starves every phase and it should not be used. The composition also surfaces a structural burden: the 1989 interfaces cannot warm-start from prior evaluations, so each phase re-acquires information the process already holds.

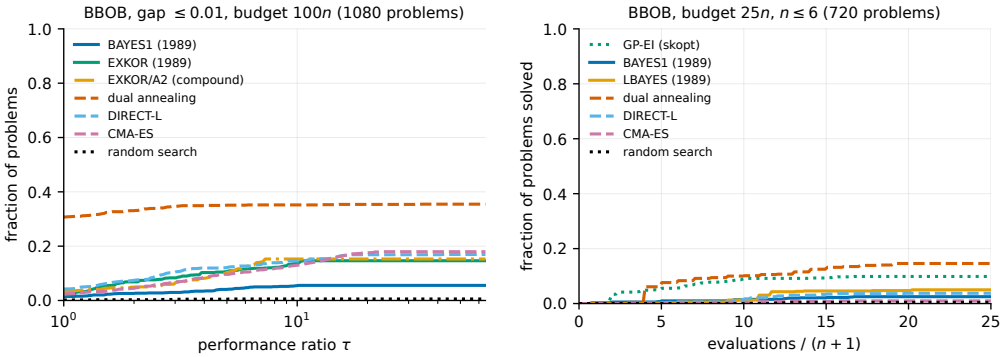


Fig. 5. BBOB suite (24 functions, $n \in \{2, 5, 10\}$, 15 instances). Left: performance profiles at gap $10^{-2}$, budget $100n$. Right: tight-budget data profile ($25n$, $n \leq 6$) including the GP-EI baseline.

*Deployed fidelity check.* The repaired Rust translation of BAYES1 is statistically indistinguishable from the wrapped original across the sweep: solve rates 15.6% versus 15.8% and only 7 of 480 instances with discordant outcomes (McNemar $p = 1$; 23 of 1,080 on BBOB, $p = 0.21$)—the benchmark-scale confirmation of the trace-level parity of Section 4. The wrapped Fortran methods themselves replicate bitwise between the R and Python harnesses on identical instances, which is what exposed both the upstream uninitialized-variable defect of Section 2.2 and the harness defect discussed under threats.

*Baselines and sanity checks.* Random search solves 0.6% of instances at $100n$; every method clears it comfortably. Differential evolution and GA suffer from population sizing at these budgets ($10n$–50 individuals leave few generations); we report them under the same budget-scaled defaults as everything else rather than tuning per problem.

### 6.2 External validation on BBOB

The BBOB replication (Figure 5) separates what generalizes from what was structure-specific. Three findings replicate. Dual annealing remains the overall winner (35.5% of instances at $100n$, ahead of SHGO at 30.0%; 14.6% at $25n$, $n \leq 6$). GP-EI remains well ahead of its 1989 ancestors at tight budgets (9.9% versus 5.3% for EXKOR/A2 and 5.0% for LBAYES). And the repaired Rust translation of BAYES1 remains statistically indistinguishable from the wrapped Fortran (23 of 1,080 instances discordant, $p = 0.21$). One finding is properly cut down to size: EXKOR, statistically tied with dual annealing on the classical set, drops to sixth (14.6%, versus 252–27 discordant instances in dual annealing's favor, $p < 10^{-15}$) — BBOB's deliberately non-separable, ill-conditioned function groups remove exactly the coordinate-aligned structure the classical test canon is rich in. The compound EXKOR/A2 edges past its parent here (15.3%, 22 versus 15 discordant, $p = 0.32$), consistent with its design target; EXKOR remains slightly behind DIRECT-L (65 versus 91, $p = 0.045$) and ahead of CRS2, ISRES, and differential evolution; BAYES1 solves 5.6%. We read the pair of suites as bracketing practice: real engineering objectives fall between the classical canon's separability and BBOB's adversarial conditioning, and the 1989 methods' standing moves accordingly.

### 6.3 Overheads and throughput

With cheap analytic objectives, wall time exposes solver internals. The wrapped 1989 methods occupy the fastest tier: median time per $100n$-budget run is 1.8 ms for EXKOR, 3.8 ms for the compound

EXKOR/A2, 2.5–2.7 ms for MIG2 and GLOPT—at or below the 2.8 ms of a bare random-search loop in Python, i.e. below the interpreter cost of the objective itself. NLopt methods run at 5–7 ms, SciPy DIRECT at 7 ms, while dual annealing (38 ms), differential evolution (42 ms), and CMA-ES (107 ms) carry their library-internal overheads, and BAYES1 (79 ms Python, 102 ms R) spends its time in the $O(\ell^2 n)$ acquisition scan. Combined with Section 4: for objectives costing under a microsecond the callback into the host language dominates everything else, and the built-in compiled-objective path of our packages removes even that.

## 7 DISCUSSION

*Wrap or rewrite?* Our evidence favors *wrap first, rewrite selectively, and never rewrite without the wrapped original in the loop.* Wrapping delivered the genuine algorithms at compiled speed in weeks of effort dominated by interface archaeology, not numerics; the fidelity incidents of Section 4 were each invisible to code review and immediate under trajectory-parity testing. The repaired translation is not redundant: it removes the 1989 array caps and COMMON-block thread-unsafety, and compiles to platforms without a Fortran toolchain. The two routes are complements—but only the executable original can arbitrate the translation's fidelity.

*Legacy algorithms, modern context.* The acquisition criterion (1) costs $O(\ell n)$ per candidate against the $O(\ell^3)$ of GP-based acquisition, and the measured throughput reflects it: the wrapped methods run at or below the cost of a bare interpreter loop over the objective. The benchmark places the 1989 methods squarely in the modern mid-field—EXKOR at the top on separable landscapes and even overall at the $100n$ budget, LBAYES respectable at tight budgets, BAYES1 ahead of several methods a generation younger—while GenSA and dual annealing, which postdate the library by a decade or more, are the consistent overall winners. For practitioners the packages are thus more than a museum: on separable or near-separable problems EXKOR is a strong and extremely inexpensive choice, and the library's LP-$\tau$ designs and screening routines remain useful building blocks. Against its direct descendant, GP-based expected improvement, the comparison quantifies the field's progress cleanly: the modern acquisition solves 6.5× as many tight-budget instances as BAYES1 while spending nearly four orders of magnitude more time per run—each side of that trade-off still has its regime.

*Scope: what the 1989 artifact cannot address.* Our experiments stop at $n = 10$ continuous box-constrained problems because the artifact does: the working arrays cap most methods at $n \leq 20$ and around $10^3$ evaluations, and the interfaces know nothing of categorical, conditional, or batch-parallel evaluation. The dominant global-optimization workload of the past decade—configuration and hyperparameter search for deep networks—is high-dimensional, mixed discrete-continuous, conditional, and noisy, and is served by purpose-built machinery such as TPE and SMAC [1, 10]; wrapping a 1989 library cannot and should not compete there, and we make no such claim. Two historical footnotes are worth recording, though: LBAYES already supports leading integer variables (handled by interpolation between adjacent integer points), and its stochastic-approximation step schedule was designed for noisy objectives—the concerns of modern configuration search were visible in 1989, at 1989 scale.

*Recommendations.* The measured evidence supports concrete guidance. (1) Under generous budgets on inexpensive continuous objectives, dual annealing (Python) and GenSA (R) are the strongest defaults we measured; DIRECT-L is the strongest deterministic choice. (2) Under tight budgets with expensive objectives, GP-based expected improvement is worth its solver cost as long as one solver step is cheaper than one objective evaluation; below that, dual annealing again. (3) On problems known or suspected to be separable or coordinate-aligned, EXKOR delivers top-tier quality at the

lowest solver cost we measured (below a bare interpreter loop)—and its failure mode is legible: it loses exactly when the landscape couples coordinates. If total failures are costlier than average regressions, its compound variant `EXKOR/A2` trades a few points of aggregate solve rate for substantially fewer zero-solve families. (4) For modernizers: wrap first; translate only with the original in the test loop asserting full-trajectory parity; treat every branch of legacy control flow as load-bearing until proven otherwise.

*Toward compound algorithms.* The compound question is answered quantitatively in Section 6: composing the library's own parts (LP-$\tau$ + `ANAL2` + `EXKOR` under an influence-ratio guard) yields robustness — four of five total-failure families repaired, better mean ranks in both ecosystems, a nominal edge on BBOB — but not aggregate throughput, and the design iterations that led to the guard and the portfolio split are themselves evidence of how easily naive composition destroys the very structure a coordinate method exploits. A second composition we have not yet built follows the same logic at the acquisition level: use the $O(\ell n)$ criterion (1) to prescreen thousands of candidates and reserve the $O(\ell^3)$ Gaussian-process posterior for the shortlist — a budget-matched bridge between the 1989 trade and the modern one. The deeper obstacle either way is interface, not mathematics: the 1989 routines cannot ingest prior evaluations, so every composed phase repurchases information the process already holds.

*Threats to validity.* Single host, cheap analytic objectives (time comparisons expose overheads, not expensive-objective regimes); default-leaning hyperparameters for competitors (budget-scaled population sizes; no per-problem tuning for any method, including ours—the wrapped methods use the upstream example parameters with budget-scaled evaluation counts); the 1989 caps censor high-budget behavior of the wrapped methods; microsecond-scale timings taken with background load and reported as medians of repeated runs (an idle-machine replication on a second host reproduced the post-repair Rust/Fortran ratios at 1.26/1.16/1.10 for $n = 2/10/20$). Four of the sixteen classical test families (the Shekel and Hartmann variants) also ship as demonstration objectives with the 1989 library, a potential selection bias in its favor; the seeded shifts perturb them away from the shipped coefficients, and the BBOB replication (Section 6) quantifies exactly how the conclusions move on an independently designed suite. One run per instance follows COCO practice; the paired tests operate on the 480 (custom) and 1,080 (BBOB) instances rather than on within-instance replicates. Finally, our own harness supplied a cautionary tale of exactly the kind this paper documents: an early version of the R driver used `pi` as a loop index, silently shadowing R's built-in $\pi$ constant and corrupting every trigonometric test objective evaluated after the first iteration—the sweep remained internally consistent (all methods saw the same corrupted functions) and was exposed only by the cross-language identity checks, after which all R results were regenerated. Redundant implementations that must agree bitwise catch what internal consistency cannot.

## 8 RELATED WORK

Global-optimization benchmarking methodology is standardized by performance profiles [3], data profiles [19], and the COCO platform [7]; we follow all three. The historical position of the Vilnius school is surveyed in [28, 33]; modern Bayesian optimization in [5, 24]. The software route of wrapping venerable Fortran into scripting ecosystems is the founding pattern of scientific Python and R [2, 30]; our contribution is a measured head-to-head of that pattern against a memory-safe-language rewrite of the same code, with the failure modes documented.

## 9 AVAILABILITY

The R package, Python wheel sources, Rust crate, vendored and patched Fortran, benchmark harnesses, raw results, and every script producing a number or figure in this paper are available at https://github.com/audrism/globalopt. The upstream distribution remains available from [17] and is vendored verbatim (patches applied mechanically and documented) for reproducibility.

*Acknowledgments.* The original MINIMUM/GlobalMinimum library is the work of Jonas Mockus (1931–2021), the author's father; this paper is dedicated to his memory. The familial relationship is disclosed in the interest of transparency; all comparative results are produced by the scripted, seeded pipeline shipped with the repository.

## A MEDIAN TERMINAL GAPS

Table 1 lists median terminal gaps at the $100n$ budget by method: over the scalable families at each dimension, and over the fixed-dimension classics. "–" marks methods not run in that setting (SHGO above $n = 5$).

## REFERENCES

[1] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for Hyper-Parameter Optimization. In *Advances in Neural Information Processing Systems 24*. 2546–2554.

[2] John M. Chambers. 2008. *Software for Data Analysis: Programming with R*. Springer, New York. https://doi.org/10.1007/978-0-387-75936-4

[3] Elizabeth D. Dolan and Jorge J. Moré. 2002. Benchmarking Optimization Software with Performance Profiles. *Mathematical Programming* 91, 2 (2002), 201–213. https://doi.org/10.1007/s101070100263

[4] Stefan C. Endres, Carl Sandrock, and Walter W. Focke. 2018. A Simplicial Homology Algorithm for Lipschitz Optimisation. *Journal of Global Optimization* 72, 2 (2018), 181–217. https://doi.org/10.1007/s10898-018-0645-y

[5] Peter I. Frazier. 2018. A Tutorial on Bayesian Optimization. (2018). arXiv:1807.02811 [stat.ML]

[6] Jörg M. Gablonsky and Carl T. Kelley. 2001. A Locally-Biased Form of the DIRECT Algorithm. *Journal of Global Optimization* 21, 1 (2001), 27–37. https://doi.org/10.1023/A:1017930332101

[7] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. 2021. COCO: A Platform for Comparing Continuous Optimizers in a Black-Box Setting. *Optimization Methods and Software* 36, 1 (2021), 114–144. https://doi.org/10.1080/10556788.2020.1808977

[8] Nikolaus Hansen and Andreas Ostermeier. 2001. Completely Derandomized Self-Adaptation in Evolution Strategies. *Evolutionary Computation* 9, 2 (2001), 159–195. https://doi.org/10.1162/106365601750190398

[9] Tim Head, Manoj Kumar, Holger Nahrstaedt, Gilles Louppe, and Iaroslav Shcherbatyi. 2021. scikit-optimize/scikit-optimize. Zenodo. https://doi.org/10.5281/zenodo.5565057

[10] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. 2011. Sequential Model-Based Optimization for General Algorithm Configuration. In *Learning and Intelligent Optimization (LION 5) (LNCS, Vol. 6683)*. 507–523. https://doi.org/10.1007/978-3-642-25566-3_40

[11] Steven G. Johnson. 2007. The NLopt Nonlinear-Optimization Package. https://github.com/stevengj/nlopt.

[12] Donald R. Jones, Cary D. Perttunen, and Bruce E. Stuckman. 1993. Lipschitzian Optimization Without the Lipschitz Constant. *Journal of Optimization Theory and Applications* 79, 1 (1993), 157–181. https://doi.org/10.1007/BF00941892

[13] Donald R. Jones, Matthias Schonlau, and William J. Welch. 1998. Efficient Global Optimization of Expensive Black-Box Functions. *Journal of Global Optimization* 13, 4 (1998), 455–492. https://doi.org/10.1023/A:1008306431147

[14] P. Kaelo and M. M. Ali. 2006. Some Variants of the Controlled Random Search Algorithm for Global Optimization. *Journal of Optimization Theory and Applications* 130, 2 (2006), 253–264. https://doi.org/10.1007/s10957-006-9101-0

[15] Harold J. Kushner. 1964. A New Method of Locating the Maximum Point of an Arbitrary Multipeak Curve in the Presence of Noise. *Journal of Basic Engineering* 86, 1 (1964), 97–106. https://doi.org/10.1115/1.3653121

[16] Jonas Mockus. 1989. *Bayesian Approach to Global Optimization: Theory and Applications*. Kluwer Academic Publishers, Dordrecht.

[17] Jonas Mockus. 1989. GlobalMinimum Fortran Source Distribution. https://globaloptimum.org/global/download/GlobalMinimumFortran.tar.gz.

[18] Jonas Mockus, Vytautas Tiesis, and Antanas Žilinskas. 1978. The Application of Bayesian Methods for Seeking the Extremum. In *Towards Global Optimization*. Vol. 2. North-Holland, Amsterdam, 117–129.

| Ecosystem | Method | scalable, median gap at 100$n$ | | | classics |
|---|---|---|---|---|---|
| | | $n$=2 | $n$=5 | $n$=10 | |
| Python | dual annealing | 2.1e-12 | 1.7e-1 | 1.1e-1 | 6.5e-13 |
| Python | DIRECT-L | 6.1e-6 | 9.8e-1 | 4.1e+00 | 2.4e-6 |
| Python | CMA-ES | 4.0e-5 | 1.1e-1 | 5.1e-1 | 4.5e-5 |
| Python | EXKOR (1989) | 4.6e-5 | 2.5e-5 | 1.9e-5 | 7.7e-1 |
| Python | SHGO | 4.6e-5 | 1.6e+00 | – | 3.2e-14 |
| Python | DIRECT (SciPy) | 1.3e-4 | 1.2e+00 | 6.0e+00 | 4.8e-5 |
| Python | MLSL | 2.4e-3 | 1.3e+00 | 2.0e+00 | 2.7e-4 |
| Python | EXKOR/A2 (compound) | 6.0e-3 | 1.9e-1 | 1.2e-1 | 6.6e-3 |
| Python | CRS2 | 5.3e-2 | 3.3e+00 | 8.0e+00 | 2.1e-1 |
| Python | diff. evolution | 1.1e-1 | 7.2e+00 | 3.7e+01 | 4.2e-1 |
| Python | GLOPT (1989) | 1.3e-1 | 3.4e+00 | 9.5e+00 | 2.6e-1 |
| Python | BAYES1 (1989) | 1.5e-1 | 6.9e+00 | 1.2e+01 | 3.5e-1 |
| Python | BAYES1 (Rust) | 1.5e-1 | 7.3e+00 | 1.2e+01 | 3.9e-1 |
| Python | MIG2 (Rust) | 2.6e-1 | 1.8e+01 | 6.8e+01 | 8.2e-1 |
| Python | LPMIN (1989) | 3.2e-1 | 7.1e+00 | 1.2e+01 | 1.0e+00 |
| Python | LBAYES (1989) | 3.4e-1 | 1.1e+00 | 2.8e+00 | 3.2e-1 |
| Python | UNT (1989) | 3.6e-1 | 1.5e+01 | 7.7e+01 | 1.1e+00 |
| Python | ISRES | 4.3e-1 | 6.9e+00 | 1.2e+01 | 9.2e-1 |
| Python | MIG2 (1989) | 5.6e-1 | 1.7e+01 | 7.4e+01 | 1.0e+00 |
| Python | random search | 6.2e-1 | 1.6e+01 | 7.1e+01 | 1.5e+00 |
| R | GenSA | 5.3e-15 | 9.0e-2 | 2.7e-1 | 3.8e-15 |
| R | DIRECT-L | 6.6e-6 | 9.8e-1 | 4.1e+00 | 2.7e-6 |
| R | EXKOR (1989) | 4.6e-5 | 2.4e-5 | 1.9e-5 | 7.7e-1 |
| R | EXKOR/A2 (compound) | 6.0e-3 | 2.5e-1 | 1.2e-1 | 6.6e-3 |
| R | CRS2 | 4.8e-2 | 3.5e+00 | 9.4e+00 | 2.1e-1 |
| R | DEoptim | 1.1e-1 | 9.5e+00 | 4.5e+01 | 5.8e-1 |
| R | GLOPT (1989) | 1.3e-1 | 3.4e+00 | 9.5e+00 | 2.6e-1 |
| R | BAYES1 (1989) | 1.5e-1 | 6.9e+00 | 1.2e+01 | 3.5e-1 |
| R | LPMIN (1989) | 3.2e-1 | 7.1e+00 | 1.2e+01 | 1.0e+00 |
| R | GA | 3.2e-1 | 4.8e+00 | 8.5e+00 | 7.8e-1 |
| R | LBAYES (1989) | 3.4e-1 | 1.1e+00 | 2.8e+00 | 3.2e-1 |
| R | UNT (1989) | 3.6e-1 | 1.5e+01 | 7.7e+01 | 1.1e+00 |
| R | ISRES | 4.0e-1 | 7.2e+00 | 1.2e+01 | 9.1e-1 |
| R | MIG2 (1989) | 5.6e-1 | 1.7e+01 | 7.4e+01 | 1.0e+00 |
| R | random search | 7.0e-1 | 1.6e+01 | 7.9e+01 | 1.0e+00 |

Table 1. Median gap to the global optimum at budget 100$n$.

[19] Jorge J. Moré and Stefan M. Wild. 2009. Benchmarking Derivative-Free Optimization Algorithms. *SIAM Journal on Optimization* 20, 1 (2009), 172–191. https://doi.org/10.1137/080724083

[20] Katharine M. Mullen, David Ardia, David L. Gil, Donald Windover, and James Cline. 2011. DEoptim: An R Package for Global Optimization by Differential Evolution. *Journal of Statistical Software* 40, 6 (2011), 1–26. https://doi.org/10.18637/jss.v040.i06

[21] Alexander H. G. Rinnooy Kan and Gerrit T. Timmer. 1987. Stochastic Global Optimization Methods, Part II: Multi Level Methods. *Mathematical Programming* 39, 1 (1987), 57–78. https://doi.org/10.1007/BF02592071

[22] Thomas P. Runarsson and Xin Yao. 2000. Stochastic Ranking for Constrained Evolutionary Optimization. *IEEE Transactions on Evolutionary Computation* 4, 3 (2000), 284–294. https://doi.org/10.1109/4235.873238

[23] Luca Scrucca. 2013. GA: A Package for Genetic Algorithms in R. *Journal of Statistical Software* 53, 4 (2013), 1–37. https://doi.org/10.18637/jss.v053.i04

[24] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. 2016. Taking the Human Out of the Loop: A Review of Bayesian Optimization. *Proc. IEEE* 104, 1 (2016), 148–175. https://doi.org/10.1109/JPROC.2015.2494218

[25] Ilya M. Sobol'. 1967. On the Distribution of Points in a Cube and the Approximate Evaluation of Integrals. *U. S. S. R. Comput. Math. and Math. Phys.* 7, 4 (1967), 86–112. https://doi.org/10.1016/0041-5553(67)90144-9

[26] Rainer Storn and Kenneth Price. 1997. Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces. *Journal of Global Optimization* 11, 4 (1997), 341–359. https://doi.org/10.1023/A:1008202821328

[27] Sonja Surjanovic and Derek Bingham. 2013. Virtual Library of Simulation Experiments: Test Functions and Datasets. https://www.sfu.ca/~ssurjano/.

[28] Aimo Törn and Antanas Žilinskas. 1989. *Global Optimization*. Lecture Notes in Computer Science, Vol. 350. Springer. https://doi.org/10.1007/3-540-50871-6

[29] Constantino Tsallis and Daniel A. Stariolo. 1996. Generalized Simulated Annealing. *Physica A: Statistical Mechanics and its Applications* 233, 1–2 (1996), 395–406. https://doi.org/10.1016/S0378-4371(96)00271-3

[30] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. https://doi.org/10.1038/s41592-019-0686-2

[31] Yang Xiang, Sylvain Gubian, Brian Suomela, and Julia Hoeng. 2013. Generalized Simulated Annealing for Global Optimization: The GenSA Package. *The R Journal* 5, 1 (2013), 13–28. https://doi.org/10.32614/RJ-2013-002

[32] Y. Xiang, D. Y. Sun, W. Fan, and X. G. Gong. 1997. Generalized Simulated Annealing Algorithm and Its Application to the Thomson Model. *Physics Letters A* 233, 3 (1997), 216–220. https://doi.org/10.1016/S0375-9601(97)00474-X

[33] Antanas Žilinskas and Anatoly Zhigljavsky. 2016. Stochastic Global Optimization: A Review on the Occasion of 25 Years of Informatica. *Informatica* 27, 2 (2016), 229–256. https://doi.org/10.15388/Informatica.2016.83